PROYECTO INTEGRADOR · MLOps

Plan de trabajo por integrante

MiBici · Predicción de viajes, riesgo operativo y agrupación de estaciones

PROPÓSITO Organizar la primera entrega del proyecto entre cuatro personas, con tareas ejecutables, responsables, productos verificables y criterios comunes de integración.

Distribución general

Rol	Enfoque	Producto principal
Integrante 1	Infraestructura, ingesta y versión de datos	Repositorio ccds, dataset.py, config.py y manifiesto
Integrante 2	P1 · Regresión de demanda	Features P1, baseline y run P1 en MLflow
Integrante 3	P2 · Clasificación del riesgo aproximado	Definición de etiqueta, features P2 y run P2
Integrante 4	P3 · Clustering + coordinación de arquitectura	Perfiles de estaciones, run P3 y diagrama

Datos comunes y alcance acordado

- Fuente: archivos mensuales de viajes MiBici y archivo de nomenclatura de estaciones (id, coordenadas y capacidad).
- Periodo propuesto: octubre de 2025 a septiembre de 2026 (12 meses); sujeto a verificar disponibilidad, integridad y correspondencia de estaciones.
- Grano inicial para P1/P2: estación × hora. Para P3: perfil resumido por estación.
- Partición cronológica propuesta: entrenamiento oct-2025 a jun-2026; validación jul-2026; prueba ago-sep-2026.

DISTINCIÓN IMPORTANTE El documento del profesor exige decisiones justificadas y evidencia; las fechas, métricas y algoritmos concretos que aparecen aquí son propuestas del equipo por validar con los datos reales.

Documento operativo para organizar el trabajo. Reemplazar "Integrante 1–4" por nombres, roles y número de equipo antes de entregarlo o distribuirlo.

1. Acuerdos técnicos que deben aprobar los cuatro

Preguntas y salidas esperadas

Pregunta	Qué se obtiene	Baseline recomendado	Métrica guía
P1 · Regresión	Salidas en la próxima hora, por estación	Predicción histórica de la misma hora de la semana previa	MAE en prueba; RMSE como apoyo
P2 · Clasificación	Indicador de presión neta elevada en hora siguiente; no disponibilidad real	DummyClassifier / Logistic Regression como candidato	Recall y F1 en prueba
P3 · Clustering	Grupos de estaciones según perfiles de uso	K-Means con variables normalizadas	Silhouette + interpretación

Variables compartidas

- A partir de viajes: inicio/fin, Origen_Id, Destino_Id; construir salidas y llegadas por estación-hora, balance = llegadas − salidas y series con horas de cero viajes para estaciones activas.
- A partir de estaciones: id, name, latitude, longitude, dpcapacity. Validar compatibilidad histórica: el archivo de septiembre de 2026 no necesariamente describe la capacidad de meses anteriores.
- Variables predictoras disponibles al corte: calendario de la hora objetivo, rezagos de salidas/llegadas (1 h, 3 h, 24 h), misma hora 7 días antes, promedios móviles y capacidad validada.
- Descartar identificadores personales y datos demográficos del baseline salvo justificación concreta; documentar tratamiento de Usuario_Id y columnas sensibles.

Definición preliminar de P2

Definir para estación s y hora futura t+1: presión = (salidas futuras − llegadas futuras) / capacidad de la estación. Etiquetar “presión elevada” cuando el valor absoluto supere un umbral aprendido/justificado con datos de entrenamiento (0.25 es solo un candidato exploratorio). Conservar el signo para diferenciar presión por salidas y por llegadas.

ADVERTENCIA METODOLÓGICA La capacidad máxima NO permite inferir la ocupación inicial ni los movimientos del personal de redistribución. En P2 se estima presión operativa, no un evento comprobado de quedarse sin bicicletas o anclajes.

Regla de fuga de información

Para predecir la hora t+1, ninguna característica puede incluir viajes de t+1 o posteriores; la etiqueta sí usa la observación futura. Ajustar imputación, escalado, umbrales y modelos únicamente con entrenamiento. Respetar los cortes cronológicos.

2. Integrante 1 | Infraestructura, datos y reproducibilidad

Misión: dejar una base técnica común para que los otros tres integrantes no descarguen ni limpien datos de formas distintas.

Pasos en orden

1. Acordar con el equipo número de equipo, nombre de módulo, nombres de variables, periodo y reglas de carpeta; registrar decisiones en README.
2. Generar estructura mediante Cookiecutter Data Science (ccds; include_code_scaffold = Yes) e inicializar el repositorio TC5061-Proyecto-EquipoNN.
3. Preparar .gitignore para data/, mlruns/, mlartifacts/, modelos serializados, .env, credenciales y cachés; prohibir datos MiBici en historial Git.
4. Centralizar rutas, semilla, periodo y parámetros compartidos en config.py del módulo. Fijar dependencias y versiones.
5. Implementar dataset.py ejecutable desde Terminal: descarga de meses y nomenclatura; detección de codificación; normalización de columnas/tipos; fechas y zonas horarias.
6. Auditar y documentar faltantes, duplicados, registros inválidos, rangos de duración, orígenes/destinos sin correspondencia y estaciones abiertas solo parte del periodo.
7. Guardar tablas procesadas en Parquet; producir manifiesto versionado con ruta de origen, fecha de descarga, cantidad de filas, SHA-256 de cada archivo y versión global.
8. Probar una segunda ejecución de ingesta (mismo resultado o cambios explicados) y documentar el comando completo en README.
9. Abrir PR propio y revisar el PR del integrante 2. Entregar evidencias del conteo real de viajes, meses y estaciones.

Archivos y evidencias que entrega

Archivos / elementos	Criterio de aceptación
dataset.py + config.py	Una orden de Terminal reconstruye los datos sin pasos manuales.
Parquet local + manifiesto Git	Conteos y hashes verificables; sin CSV/Parquet pesados subidos al repo.
requirements / pyproject + .gitignore	Versiones fijas y exclusiones correctas.
README (instalación/ingesta)	Otro integrante consigue ejecutar ingesta desde un clon limpio.
PR y revisión	Historial visible de autoría y revisión de otro integrante.

DEPENDENCIA Los integrantes 2, 3 y 4 pueden diseñar features y baselines antes de la ingesta definitiva, pero deben integrar y ejecutar sus corridas solo sobre la misma versión de datos validada.

3. Integrante 2 | P1 Regresión: demanda de viajes

Misión: pronosticar cuántos viajes se iniciarán en cada estación durante la próxima hora y registrar una línea base reproducible.

Pasos en orden

1. Formular P1 en el ML Canvas: objetivo, granularidad estación × hora, horizonte +1 hora, usuario operativo y decisión de reubicación/planeación.
2. Verificar con el integrante 1 las columnas y fechas del dataset normalizado; documentar estaciones activas y horas con cero salidas.
3. Construir la etiqueta viajes_siguiente_hora = salidas de la estación en la hora futura; fijar el instante de predicción.
4. Crear funciones con docstrings en features.py (o módulo P1 importado por features.py): rezagos históricos 1 h, 3 h y 24 h; misma hora 7 días antes; día/mes/hora y codificación de estación.
5. Separar cronológicamente entrenamiento, validación y prueba; las ventanas históricas solo consultan horas previas.
6. Entrenar baseline transparente: pronóstico de la misma hora de la semana anterior, con fallback documentado cuando no exista historia.
7. Calcular MAE y RMSE con prefijos por conjunto, p. ej. val_mae y test_mae; generar gráfica real vs. predicho o errores por estación.
8. Ejecutar el baseline desde script del repositorio; registrar en MLflow parámetros, versión de datos, rango de fechas, métricas, artefacto y etiquetas P1/equipo/commit.
9. Proponer candidato posterior (Random Forest Regressor o modelo equivalente) y justificar; redactar sección P1 del PDF y abrir PR con pruebas mínimas.

Entregables y evidencias

Entregable	Verificación
Features de demanda	Funciones reutilizables y documentadas; sin futuro en X.
Script baseline P1	Se ejecuta con un comando desde la raíz del repositorio.
Run P1 en MLflow	Métricas MAE/RMSE, fecha, hash de datos, git_commit y gráfica.
Sección PDF P1	Qué se predice, por qué importa, quién lo usa y limitaciones.
Pull Request	Revisado por integrante 3.

PRUEBA CLAVE Comparar contra una predicción simple: si el baseline falla en estaciones sin historial, usar una regla de respaldo explícita, nunca completar con información futura.

4. Integrante 3 | P2 Clasificación: presión operativa

Misión: construir una aproximación auditable al riesgo de falta de bicicletas o anclajes, sin afirmar que los viajes miden ocupación real.

Pasos en orden

1. Formular P2 en ML Canvas: estación × hora, evento futuro, relevancia para reubicación y persona que utilizará la alerta.
2. Construir series de salidas (fecha de inicio) y llegadas (fecha de fin) por estación-hora. Asegurar cobertura de horas sin movimientos y capacidad válida.
3. Calcular presión futura = (salidas − llegadas de la siguiente hora) / capacidad. Conservar dirección: salidas dominantes o llegadas dominantes.
4. Analizar en entrenamiento la distribución real de la presión normalizada y la proporción de clases para umbrales candidatos (p. ej., 0.25); elegir y justificar sin mirar prueba.
5. Definir etiqueta binaria “presión operativa elevada” y documentar que es una proxy, no quiebre de inventario observado. Explicar supuestos y efectos de rebalanceo no observado.
6. Crear features históricas: salidas/llegadas de última hora, sumas 3/24 h, balance previo/capacidad, calendario, capacidad y variables conocidas en t.
7. Entrenar DummyClassifier como baseline mínimo; comparar o proponer Logistic Regression / Random Forest como candidato. Considerar desbalance.
8. Evaluar F1 y recall de la clase positiva, además de precision, matriz de confusión y proporción de alertas; registrar métricas separadas val_/test_.
9. Ejecutar desde script en MLflow con datos/fechas/versiones, etiquetas equipo/P2/commit, artefactos y justificación de umbral; abrir PR y documentar en PDF.

Entregables y evidencias

Entregable	Verificación
Especificación de etiqueta P2	Fórmula, umbral elegido, prevalencia y limitaciones.
Features de presión histórica	Variables calculadas antes de la hora objetivo.
Baseline y matriz de confusión	Evaluación interpretable de clase positiva.
Run P2 en MLflow	Métricas, artefacto, rango y versión de datos, commit.
Pull Request	Revisado por integrante 4.

RIESGO A CONTROLAR Ni la capacidad ni el balance neto garantizan conocer disponibilidad. El documento final debe nombrar al objetivo “riesgo aproximado por presión operativa” y no “estación vacía/llena confirmada”.

5. Integrante 4 | P3 Clustering, arquitectura e integración

Misión: identificar perfiles de estaciones e integrar la propuesta global, sin asumir que la geografía determina por sí sola los grupos.

Pasos en orden

1. Formular P3 en el ML Canvas: unidad estación; ventanas históricas para formar perfiles; destinatario y decisiones de operación/planeación.
2. Coordinar con integrante 1 el conjunto de estaciones observadas y las coordenadas; verificar que no se atribuyan estaciones actuales a periodos anteriores por error.
3. Crear en features.py funciones de agregación por estación: viajes promedio diario, demanda por hora, % en horas pico, % fin de semana, relación llegadas/salidas y rotación por capacidad.
4. Manejar valores faltantes y escalar características numéricas usando solo la ventana de análisis definida. No introducir futuro en perfiles históricos.
5. Implementar K-Means como baseline, documentando selección inicial de k y random_state. Proponer clustering jerárquico como alternativa de mejora.
6. Calcular Silhouette cuando sea válido; examinar tamaños, perfiles promedio y estabilidad de grupos. Generar gráfico de perfiles y mapa de estaciones con colores por cluster.
7. Registrar en MLflow run P3 con k, ventana, versión de datos, métrica, artefactos y etiquetas equipo/P3/commit.
8. Diseñar el diagrama de arquitectura: CSV oficiales → ingesta → Parquet/manifest → features → modelos → MLflow y reportes; explicar herramientas.
9. Coordinar la integración del ML Canvas, README y PDF (sin hacer solo toda la redacción), abrir PR propio y revisar el PR del integrante 1.

Entregables y evidencias

Entregable	Verificación
Funciones de perfiles	Estación = unidad de observación; datos históricos definidos.
Run P3 y gráficos	Silhouette (si aplica), distribuciones de grupos y mapa.
Diagrama en references/	Herramientas y conexiones explícitas.
ML Canvas consolidado + sección PDF	P1, P2 y P3 consistentes con implementación.
Pull Request	Revisado por integrante 1.

CRITERIO DE INTERPRETACIÓN La ubicación debe mostrarse en un mapa; no incluir latitud/longitud como variables dominantes en el primer clustering, salvo que se justifique su efecto.

6. Trabajo compartido: GitHub, MLflow y evidencia

Reglas de integración en GitHub

Responsable	Rama de trabajo	Revisión asignada
Integrante 1	feature/data-ingestion	Revisa PR de P3
Integrante 2	feature/p1-regression	Revisa PR de ingesta
Integrante 3	feature/p2-classification	Revisa PR de P1
Integrante 4	feature/p3-clustering	Revisa PR de P2

- La rama main debe estar protegida: prohibir cambios directos y exigir revisión antes de integrar PR. Cada integrante: ≥1 PR propio, ≥1 revisión ajena y commits identificables.
- Convención sugerida: feature/p1-regression y commits como feat(p1): add hourly lag features; fix(data): normalize station ids.
- Asegurar que el repositorio excluye datos de MiBici, artefactos locales de MLflow, modelos serializados y credenciales, incluso del historial.

Contrato común de MLflow

Un experimento único denominado, por ejemplo, mibici-equipoNN; al menos un run por P1, P2 y P3 ejecutado desde scripts versionados. Etiquetas y nombres de métricas consistentes, p. ej. train_*, val_* y test_*.

Registrar en cada run	Contenido esperado
Parámetros	Pregunta, modelo, semilla, fechas, granularidad, horizonte, versión/hash de datos e hiperparámetros.
Métricas	P1: val_mae/test_mae; P2: val_f1/test_f1 y recall; P3: silhouette o justificación si no aplica.
Tags	team, question (P1/P2/P3), git_commit; nombre del run descriptivo.
Artefactos	P1 gráfica demanda; P2 matriz de confusión; P3 mapa/perfiles; sin información personal.

Cuatro capturas obligatorias para el PDF

- Lista de runs del experimento común (P1, P2 y P3).
- Detalle de un run con parámetros, métricas y etiquetas.
- Visualización de al menos un artefacto registrado.
- Comparación de dos o más runs en MLflow (aunque sean tareas distintas, explicar que sus métricas no son directamente comparables).

Si se desean comparar desempeños cuantitativamente, conviene añadir una segunda corrida del mismo problema, además de cumplir el mínimo de una por pregunta.

7. Calendario por hitos y dependencias

Secuencia sugerida sin fechas de entrega inventadas. El equipo puede asignar un día límite interno a cada hito.

Hito	Responsable principal	Salida verificable / condición para avanzar
H0 · Acuerdos	Todos	Número de equipo, roles/nombres, periodo, reglas de datos, definición final P1/P2/P3.
H1 · Repositorio	Integrante 1	Cookiecutter, PR inicial y reglas de protección y revisión.
H2 · Dataset	Integrante 1	Comando ingesta, 12 meses verificados, Parquet, conteos y manifiesto.
H3 · Features	Integrantes 2, 3, 4	Funciones en módulo, pruebas de ausencia de fuga temporal.
H4 · Baselines	Integrantes 2, 3, 4	Tres runs en MLflow con métricas, tags, artefactos y commit.
H5 · Integración	Todos; coordina I4	README ejecutable desde clon limpio; canvas, diagrama y cuatro capturas.
H6 · Entrega	Todos	PDF con APA 7 y repositorio accesible al profesor; PR y revisiones comprobables.

Criterios para declarar el proyecto listo

- ☐ Se conocen cifras reales: meses descargados, total de viajes, número de estaciones y tasa de correspondencia con nomenclatura.
- ☐ Las tres preguntas cuentan con variable objetivo o perfil, granularidad, horizonte, usuario, baseline, candidato y métricas justificados.
- ☐ dataset.py descarga, normaliza y versiona sin pasos manuales; se demuestra con un clon limpio.
- ☐ features.py reutiliza funciones con docstrings; no hay fuga de datos futuros.
- ☐ Los tres runs están registrados con fechas, versión de datos, commit, parámetros, métricas, etiquetas y artefactos.
- ☐ El repositorio ccds usa config.py, dependencias fijas, README completo y .gitignore; no contiene datos ni modelos locales.
- ☐ La rama main está protegida; cuatro integrantes aportaron commits, PR y revisión.
- ☐ El PDF incluye ML Canvas, formulación P1–P3, metodología, arquitectura, cuatro capturas y referencias APA 7.

PRIMERA SESIÓN DE TRABAJO Antes de programar, completar H0: asignar nombres a integrantes, confirmar equipo NN y descargar una muestra para validar los encabezados, años y capacidad; después comenzar H1–H2.